PEA – projekt nr 1/2026

Temat: Implementacja i analiza efektywności algorytmu losowego, zachłannego - Nearest Neighour (NN) i Repetitive Nearest Neighbour (RNN) oraz Brute-Force (przegląd zupełny)

Należy zaimplementować oraz dokonać analizy efektywności w/w dla asymetrycznego problemu komiwojażera (ATSP).

Podczas realizacji zadania należy przyjąć następujące założenia:

- używane struktury danych powinny być alokowane dynamicznie (w zależności od aktualnego rozmiaru problemu),
- zakładamy, że odległości są liczbami całkowitymi
- program powinien umożliwić weryfikację poprawności działania algorytmu. W tym celu powinna istnieć możliwość wczytania danych wejściowych z pliku tekstowego,
- po zaimplementowaniu i sprawdzeniu poprawności działania algorytmu należy:
    - dla algorytmu Brute-Force (BF) dokonać pomiaru czasu jego działania w zależności od rozmiaru problemu N (badania należy wykonać dla minimum 7 różnych reprezentatywnych wartości N – należy skupić się na większych wartościach N) – dla każdej z metod wartość N powinna być różna – chodzi o to aby dla zadanego czasu można było określić liczbę N. Proszę zmierzyć wartość N dla czasu równego 2 minuty.
    - dla pozostałych algorytmów dla wartości N=10,11,12,13,14 wygenerować po 100 tablic odległości i dla każdej wygenerowanej tablicy:
        - uruchomić algorytm BF, aby obliczyć długość optymalnej ścieżki
        - uruchomić algorytm NN i RNN – obliczyć błąd względy rozwiązania
        - uruchomić algorytm losowy dla ilości permutacji = 10N i obliczyć błąd względny rozwiązania
        - dla każdego algorytmu dla danej wartości N obliczyć względny błąd średni

- implementacje algorytmów należy dokonać zgodnie z obiektowym paradygmatem programowania,
- używanie „okienek” nie jest konieczne i nie wpływa na ocenę (wystarczy wersja konsolowa),
- kod źródłowy powinien być komentowany.

Sprawozdanie powinno zawierać:

- wstęp teoretyczny zawierający opis rozpatrywanego problemu, oraz oszacowanie jego złożoności obliczeniowej na podstawie literatury dla badanych algorytmów,

2026-03-08

- krótki opis algorytmów (można wspomagać się pseudokodem) – przedstawić wady i zalety
- plan eksperymentu (rozmiar używanych struktur danych, sposób generowania danych, metoda pomiaru czasu, itp.),
- wyniki eksperymentów:
    a. dla algorytmu BF przedstawić w postaci tabel i wykresów zależność czasu wykonania algorytmu od ilości miast (N)
    b. dla pozostałych algorytmów (każdego indywidualnie) przedstawić w postaci tabel i wykresów wartość uśrednionego błędu względnego w zależności od ilości miast (wykres zrobić dla w/w algorytmów wspólny)
- wnioski dotyczące otrzymanych wyników,
- kod źródłowy w formie elektronicznej wraz z wersją wykonywalną programu (kopiowany na dysk google'a po akceptacji prowadzącego).

Uwagi do sprawozdania i programu

a) algorytm losowy - odpowiedzieć na pytanie, czy istnieje metoda generowania kolejnych permutacji w taki sposób, aby możliwość powtórzenia permutacji była minimalna, przy czym nie może to być metoda generowania kolejnych permutacji, czyli musi zostać zachowany losowy charakter tworzenia permutacji.
b) w algorytmie RNN należy uwzględnić możliwość istnienia dla każdego węzła krawędzi o takich samych kosztach (wagach), co oznacza, że należy sprawdzić każdą z możliwości wyboru, a nie tylko jedną wybraną. W praktyce należy zapamiętać taki węzeł i dokończyć poszukiwanie ścieżki z tego węzła, tyle razy, ile krawędzi o tej samej wadze jest dostępnych.
c) spróbować odpowiedzieć na pytanie - jak na złożoność obliczeniową rozwiązania TSP, dla zbadanych algorytmów:
    - wpływa gęstość grafu?
    - istnienie jednej lub wielu ścieżek o optymalnym koszcie?
    - zakresu wag krawędzi w instancji?
d) przy metodzie przeglądu zupełnego proszę nie skracać przebiegu algorytmu poprzez pomijanie niektórych permutacji. Nie jest dozwolone jest przerwanie sumowania przy obliczaniu drogi dla konkretnego rozwiązania jeśli dotychczasowy fragment drogi jest gorszy od dotychczasowego najlepszego rozwiązania – należy zrobić klasyczny przegląd zupełny.

2026-03-08

Sprawdzenie poprawności zaimplementowanego algorytmu:

Aby sprawdzić poprawność działania algorytmu musi być możliwość wczytania danych z pliku i tekstowego i wykonania na nich obliczeń. Menu programu powinno umożliwiać

1.Wczytanie danych z pliku

2.Wygenerowanie danych losowych

3.Wyświetlenie ostatnio wczytanych lub wygenerowanych danych

4.Uruchomienie danego algorytmu dla ostatnio wczytanych lub wygenerowanych danych i wyświetlenie wyników (należy wyświetlić długość ścieżki, ciąg wierzchołków oraz **czas wykonania algorytmu**). W przypadku algorytmu losowego należy zapytać o ilość permutacji.

Dane dla których będzie testowana poprawność algorytmu umieszczone są na stronie prowadzącego i/lub na dysku google.

Format danych w pliku jest następujący:

- w pierwsze linii jest podana ilość miast,
- w pozostałych liniach macierz kosztów: w każdej linii wiersz macierzy (liczby przedzielone spacją),
- dane na przekątnej mają wartość równą -1.

Ocena (maksimum):

5,0 – 4 algorytmy bez korzystania z bibliotek, w przypadku BF także rekurencji

4,0 – 4 algorytmy z możliwością korzystania z bibliotek

3,0 – Brute Force + alg. losowy

Języki programowania

Wskazane jest korzystanie z języków kompilujących się do kodu maszynowego (np. C++,Rust).

Uwaga!!!

1.Jeżeli korzystamy z Visual Studio (lub innego środowiska), to testy prowadzimy na wersji RELEASE (a nie DEBUG)

2.Należy wgrać na dysk google'a przed oddaniem wersję exe programu, gdyż w tym zadaniu algorytm BF innych osób będą testowane w tych samych warunkach – jego czas działania będzie wpływał na ocenę końcową.

2026-03-08